Stockroom API Specification

Version: 1.0
Document date: 28 October 2026
API: Node.js 24, Express 5, TypeScript, SQL Server
Base URL: http://localhost:8080 (direct) or http://localhost:8081 (same-origin portal proxy)

This document describes the API implemented by the current application. It is the maintained source for the companion API-SPECIFICATION.pdf. The browser portal proxy forwards /api requests to the API service.

Conventions and security

- JSON request bodies use Content-Type: application/json, except multipart image

uploads.

- JSON timestamps are UTC values serialized by the SQL Server driver.
- Identifiers are UUID strings. Monetary amounts are decimal numbers in the database currency; the current customer storefront labels prices as USD.
- Staff endpoints require the signed Stockroom.Auth cookie and a role permission. The cookie is HttpOnly, SameSite Strict, and lasts 30 minutes.
- Every state-changing /api request must also include X-CSRF-TOKEN matching the Stockroom.Csrf cookie. Obtain it from GET /api/auth/csrf. The CSRF cookie is readable by the first-party portal JavaScript; the auth cookie is not.
- The customer catalog is public.

Customer chat is also public, but is rate-limited to 12 requests per minute and still requires the CSRF token.

- 400 means invalid input or CSRF; 401 means missing/ invalid authentication; 403 means authenticated but missing permission; 404 means missing route/resource; 409 means a uniqueness or business-rule conflict; 429 means rate limit; 500 means an unexpected server failure; 502 means an upstream AI provider failure; 503 means the assistant is not configured.

- Error responses normally have the form {"message":"..."}. Invalid JSON returns HTTP 400 with {"messa ge":"Request body must be valid JSON."}.

- Product and order writes preserve the inventory ledger and audit trail. Sales and received purchases

update stock within the same SQL transaction.

Roles and permissions

Role	Permissions
Administrator	All permissions
Inventory Manager	products.read, products.write, inventory.read, inventory.adjust, purchases.manage, sales.manage, reports.read, audit.read
Warehouse Operator	products.read, inventory.read, inventory.adjust
Purchasing	products.read, inventory.read, purchases.manage, reports.read
Sales	products.read, inventory.read, sales.manage, reports.read
Read-only Auditor	products.read, inventory.read, reports.read, audit.read

Authentication

GET /api/

auth/status

Public. Indicates whether first-admin setup is still needed.

200

```
{ "setupRequired": false }
```

GET /api/auth/csrf

Public. Sets the Stockroom.Csrf cookie and returns its matching request token.

200

```
{ "requestToken": "base64u
```

POST /api/auth/bootstrap

Public, first-run only. Creates the first Administrator and signs the account in. Requires CSRF.

Request

```
{
  "email": "admin@example.
  "displayName": "Store Ad
  "password": "StrongPassw
}
```

Email must be valid; display name is required and at most 120 characters. Password must be at least 12 characters and include lowercase, uppercase, numeric, and symbol

characters.

201: user object (id, email, displayName, roles, permissions), Location: /api/auth/me, and auth cookie. 400: invalid fields. 409: setup already completed.

POST /api/auth/login

Public. Requires CSRF. After five failed password attempts, the account is temporarily locked for 15 minutes. A separate endpoint limiter permits at most 10 attempts per minute per client.

Request: {"email"

:"admin@exampl

e.com","passwor

d":"..."} 200: user object (id, email, displayName, roles, permissions) and auth cookie. 401: generic invalid-credentials/account-unavailable response.

GET /api/auth/me

Authenticated. Returns the current user

object: id, email, displayName, roles, permissions.

POST /api/auth/logout

Authenticated; requires CSRF. Clears the auth cookie. 204, no response body.

GET /api/auth/admin/roles

Requires users.manage. Returns {"roles":[

{"name":"Adminis

trator","permissio

ns":[...]}],"availabl

ePermissions":[...]}.

GET /api/auth/admin/users

Requires users.manage. Returns an array of user summaries with id, email, displayName, isActive, createdAtUtc, lastLoginAtUtc, and roles.

POST /api/auth/admin/users

Requires users.manage and CSRF.

```
{
  "email": "operator@examp
  "displayName": "Warehous
  "temporaryPassword": "St
```

```
"roles": ["Warehouse Operator"]
}
```

Password follows first-admin strength rules. roles must contain one or more valid role names.

201: {"id":"uuid"}. 400: validation/ unknown role. 409: email already exists.

PUT /api/auth/admin/users/{id}/roles

Requires users.manage and CSRF.

```
{ "roles": ["Sales"] }
```

At least one valid role is required. Role changes rotate the security stamp and invalidate existing sessions. The last active administrator cannot lose Administrator access. 204, 400, 404, or 409.

PUT /api/auth/admin/users/{id}/active

Requires users.manage and CSRF. Body: {"isActive":true}. Deactivation

invalidates sessions. Users cannot deactivate themselves; the last active administrator cannot be deactivated. 204, 400, 404, or 409.

Products and images

GET /api/products

Requires products.read. Returns {"items":

[Product],"totalC

ount":number,"pa

ge":number,"pag

eSize":number}. Defaults: page 1, page size 20; maximum page size 100. Query parameters:

Parameter	Meaning
search	Matches name, SKU, barcode, category, brand, or location
category	Exact category
location	Partial location
brand	Exact brand
stock	in or out
minPrice, maxPrice	Inclusive price bounds
sortBy	name, sku, barcode,

category, brand, location, price, stock

sortDirection | asc or desc

page, pageSize | Paging

Each Product contains id, name, category, price, stock, reorderLevel, barcode, sku, description, location, brand, createdAtUtc, and images. Image summaries contain id, fileName, contentType, and sortOrder.

POST /api/products

Requires products.write and CSRF. Creates a product; barcode may be omitted or empty to generate one automatically.

```
{
  "name": "Sample item",
  "category": "Accessories
  "price": 19.95,
  "stock": 12,
  "reorderLevel": 3,
  "barcode": "",
  "sku": "ACC-001",
  "description": "Product
  "location": "Aisle 2",
  "brand": "Example"
}
```

Required: non-empty name (max 120), category (max 80), finite non-negative price, non-negative integer stock. reorderLevel is a non-negative integer (defaults to 10). Barcode/SKU are max 64 characters, description 2000, location 120, brand 120. 201: Product, Location: /api/products/{id}. 400: validation. 409: duplicate barcode/SKU.

PUT /api/products/{id}

Requires products.write and CSRF. Replaces editable product fields using the same body and validation as creation. Stock changes create an inventory movement. Archived products cannot be edited. 200: Product. 400, 404, or 409.

DELETE /api/products/{id}

Requires products.write and CSRF. Archives the product; it does not physically delete it. 204 or 404.

POST /api/

products/{id}/images

Requires products.write and CSRF. multipart/form-data with one or more files fields. Up to 8 total images per product, maximum 8 MiB each. JPEG, PNG, and WebP only; image signatures are verified. 201: {"images":[{"id":"

uuid","fileName":

"photo.png","cont

entType":"image/

png","sortOrder":

0}]}. 400: invalid type, content, count, or file size. 404: missing/archived product.

GET /api/products/{id}/images/{imageId}

Requires products.read. Returns the image bytes with its stored content type. 404 when missing.

DELETE /api/products/{id}/images/{imageId}

Requires products.write and CSRF.

Deletes one gallery image. 204 or 404.

GET /api/products/pdf

Requires products.read and reports.read. Downloads products.pdf, an inventory list PDF.

GET /api/inventory/barcode/{productId}

Requires inventory.read. Returns {"format":"CODE128","value":"...","product":"...","sku":"..."}. 404 when the product is missing.

Sales and purchases

POST /api/sales

Requires sales.manage and CSRF. Completes a sale immediately; available stock is reduced. Each product may appear once; 1-100 line items. The server snapshots current product name/SKU/price and calculates

totals.

```
{
  "customerName": "Walk-ir
  "discountPercent": 5,
  "items": [
    { "productId": "uuid",
  ]
}
```

customerName is optional (max 160). Order and line discount percentages are optional and default to zero; each must be from 0 to 100. Quantity is a positive safe integer. 201: id, reference, subtotal, discountPercent, discountAmount, total, and item lines (id, productId, productName, sku, quantity, unitPrice, discountPercent, discountAmount, lineTotal, stockAfter). 400: invalid line, unavailable item, duplicate item, or insufficient stock.

GET /api/sales

Requires sales.manage. Query: page (default 1), pageSize (default 20, max 100). Returns an array of sale

summaries: id, reference, customerName, subtotal, discountPercent, discountAmount, total, itemCount, createdAtUtc.

POST /api/purchases

Requires purcha

ses.manage and CSRF. Records a completed receipt and immediately increases stock. Each product may appear once; 1-100 lines.

```
{
  "supplierName": "Supplie
  "items": [
    { "productId": "uuid",
  ]
}
```

Supplier is optional (max 160). Quantity is a positive safe integer; unit cost is finite and non-negative. 201: id, reference, total, and lines (productId, productName, quantity, unitCost, lineTotal, stockAfter). 400: invalid input, unavailable item, or stock overflow.

GET /api/purchases

Requires purcha

ses.manage. Query: page (default 1), pageSize (default 20, max 100). Returns an array of id, reference, supplierName, total, createdAtUtc.

Inventory, reports, audit

GET /api/inventory/movements

Requires inventory.read. Query: page (default 1), pageSize (default 50, max 200). Returns {"items":[Movement],"totalCount":number,"page":number,"pageSize":number}. Movement fields: id, productId, productName, sku, movementType, quantityChange, balanceAfter, reason, reference, createdByEmail, createdAtUtc. totalCount is the number of movement rows across all pages.

POST /api/inventory/adjustments

Requires inventory.adjust

and CSRF.

```
{
  "productId": "uuid",
  "quantityChange": -2,
  "reason": "Damaged stock
  "reference": "CASE-123"
}
```

Change must be a non-zero whole number; resulting stock cannot be negative or exceed SQL Server int. Reason is required (max 300). Reference is optional (max 120). 200: {"prod

uctId":"uuid","na

me":"Sample item","stock":10}. 400: invalid adjustment/ product.

GET /api/reports/dashboard

Requires reports.read. Returns product/ inventory KPIs (productCount, unitsOnHand, inv

entoryRetailValu

e, lowStockCount, o

utOfStockCount),

up to 10 low-stock items, up to 8 best-selling items, movement trend over the last 14 days, and up to 12 recent

movements.

GET /api/reports/inventory.csv

Requires reports.read. Downloads UTF-8-BOM CSV inventory.csv for active products. Cells are CSV-quoted and formula-prefixed values are neutralized for spreadsheet safety.

GET /api/reports/movements.csv

Requires reports.read. Downloads UTF-8-BOM CSV movements.csv. Cells are CSV-quoted and formula-prefixed values are neutralized for spreadsheet safety.

GET /api/audit

Requires audit.read. Query: page (default 1), pageSize (default 50, max 200). Returns {"items":[{"id":"uu

id","userId":"uuid

","userEmail":"us

er@example.co

m","action":"...","ent

ityType":"...","entityI

d":"...","details":"...",

"createdAtUtc":"..."}

],"totalCount":numb

er,"page":number,"

pageSize":number}

. totalCount is the number of audit rows across all pages.

Customer storefront and assistant

GET /api/store/products

Public. Query search (up to 100 characters; matches product name/category/brand/description/SKU), category (up to 80; exact), page (1-based; defaults to 1), and pageSize (defaults to 24 and is capped at 100). Returns {"items":[Custom

erProduct],"total

Count":number,"

page":number,"p

ageSize":number

,"categories":

["..."]} for active in-stock products. The storefront loads 24 products at a time and offers a

"Load more" action instead of loading the entire catalog in one request. Customer product fields: id, name, category, brand, description, sku, price, stock, imageId.

GET /api/store/products/{identifier}

Public. Identifier is a product UUID or an exact product name with hyphens standing in for spaces. Returns a CustomerProduct or 404 if unavailable.

GET /api/store/products/{productId}/images/{imageId}

Public. Returns image bytes for a matching active in-stock product image, cached for one hour. 404 if the image/product is invalid or unavailable.

POST /api/customer-assistant/chat

Public; CSRF required; limit 12 requests per minute. Requires

OPENAI_API_KEY in the API environment. Accepts 1-12 messages, each 1-1000 characters, with role user or assistant; the last message must be from user.

```
{
  "messages": [
    { "role": "user", "cor
  ]
}
```

The API uses semantic vector search over product catalog content in Qdrant and combines those results with bounded keyword retrieval from SQL Server. Product embeddings use OpenAI's text-embedding-3-small; the first searchable chat builds the index, and later searches refresh it at most once per minute, with product create, update, and archive operations invalidating it for the next search. Search results are reloaded from SQL Server so product fields and in-stock status are current before

they are sent to the configured OpenAI Chat Completions model. If no products match, the assistant must not recommend an invented product. Structured outputs use strict JSON Schema. This first RAG step indexes product catalog content only; help-document indexing is not yet implemented. 200:

```
{
  "customer_facing_message
  "routing_action": {
    "target_page": "produc
    "payload": {
      "search_query": null
      "product_id_or_slug"
      "category_name": nul
    }
  }
}
```

Allowed target_page: product_detail, category_listing, cart, checkout, order_status, stay_on_chat. Every payload property is required and accepts a string or null; strings are limited to 200 characters. The API validates the schema and checks product-detail IDs against retrieved products before

returning it. 400: invalid history. 429: rate limit. 502: OpenAI embedding/chat provider error or malformed model output. 503: assistant is not configured or the vector database is unavailable. Product descriptions and shopper queries are sent to OpenAI for embeddings; relevant product matches and conversation messages are sent to the configured chat provider. Do not send sensitive personal/payment information. The AI does not place orders or verify live inventory at checkout.

Operational limits and scope

- Health: GET /health executes SELECT 1 against SQL Server and returns {"status":"Healthy"}.

- Default API body limit is 1 MiB. Image upload limits are documented above.

- The storefront cart is browser memory

only. Customer checkout, payment, accounts, returns, and order lookup are not implemented.

- Orders are completed immediately through staff-only endpoints; there are no draft orders or payment capture.
- SQL Server is an external, existing database; Compose does not run or own a SQL Server container.